

Documentação Técnica

Sistema Reserva de Salas de Estudo

Ailton Baren Pereira da Silva

Julho de 2026

1 Objetivo

Este documento consolida os principais artefatos técnicos do sistema Reserva de Salas de Estudo, reunindo arquitetura, modelo de dados, requisitos, rastreabilidade e riscos em um único arquivo para facilitar a entrega acadêmica.

1.1 Escopo da Documentação

O foco deste documento é registrar as decisões técnicas e os elementos de análise e projeto que sustentam o sistema implementado. A documentação contempla a visão geral da solução, os atores do sistema, os principais fluxos, a arquitetura, o modelo de dados, as regras de negócio, a rastreabilidade dos requisitos e os riscos do projeto.

Não fazem parte deste documento detalhes de infraestrutura de produção, configuração de servidores externos, integração com sistemas acadêmicos reais ou implantação em nuvem, pois o projeto foi desenvolvido para execução local e demonstração acadêmica.

2 Diagrama de Casos de Uso

O diagrama de casos de uso apresenta as principais interações dos perfis Aluno e Administrador com o sistema.

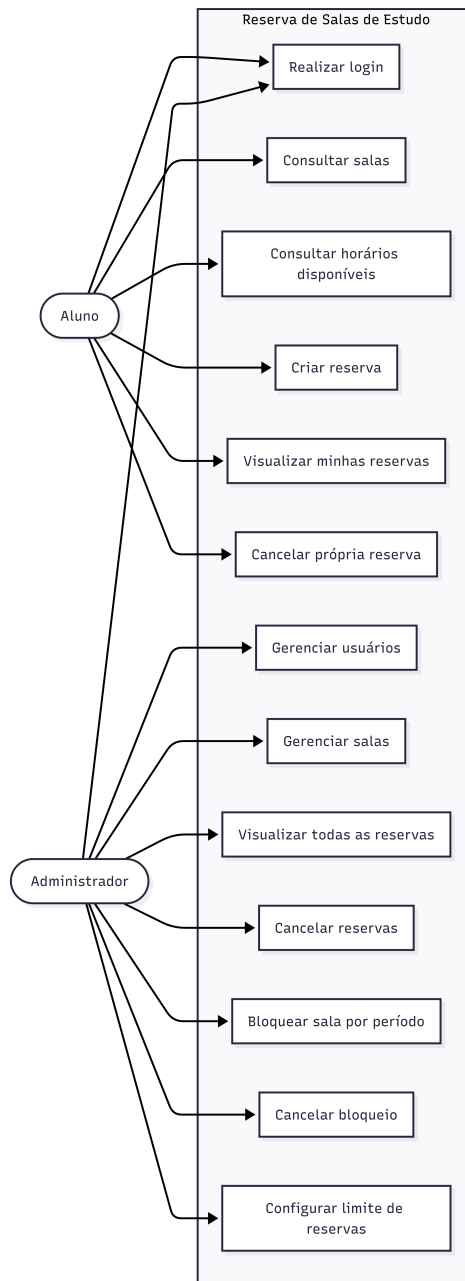


Figure 1: Diagrama de casos de uso

2.1 Atores

Ator	Descrição
Aluno	Usuário que consulta salas, verifica horários disponíveis, cria reservas, acompanha suas reservas e pode cancelar suas próprias reservas mediante informação do motivo.
Administrador	Usuário responsável pela gestão do sistema, incluindo usuários, salas, reservas, bloqueios, filtros administrativos e configurações gerais.

2.2 Principais Casos de Uso

- Realizar login e logout.
- Consultar salas cadastradas.
- Consultar horários disponíveis por sala e data.
- Criar reserva informando finalidade.
- Cancelar reserva com motivo obrigatório.
- Cadastrar e editar usuários pelo painel administrativo.
- Cadastrar, editar, desativar, bloquear e excluir salas sem histórico.
- Bloquear sala por período de dias.
- Exibir salas bloqueadas no dia atual como indisponíveis na lista de salas.
- Configurar o limite de reservas ativas por aluno.

3 Arquitetura do Sistema

3.1 Visão Geral

O sistema é uma aplicação web monolítica desenvolvida em Flask, com renderização server-side via Jinja2, persistência em PostgreSQL e interface responsiva com Bootstrap 5.

A escolha por uma arquitetura monolítica é adequada ao escopo acadêmico, pois reduz a complexidade de implantação, facilita a demonstração em ambiente local e mantém as regras de negócio concentradas em uma única aplicação. Mesmo sendo simples, o projeto separa responsabilidades em módulos de rotas, modelos, serviços, templates, arquivos estáticos e testes.

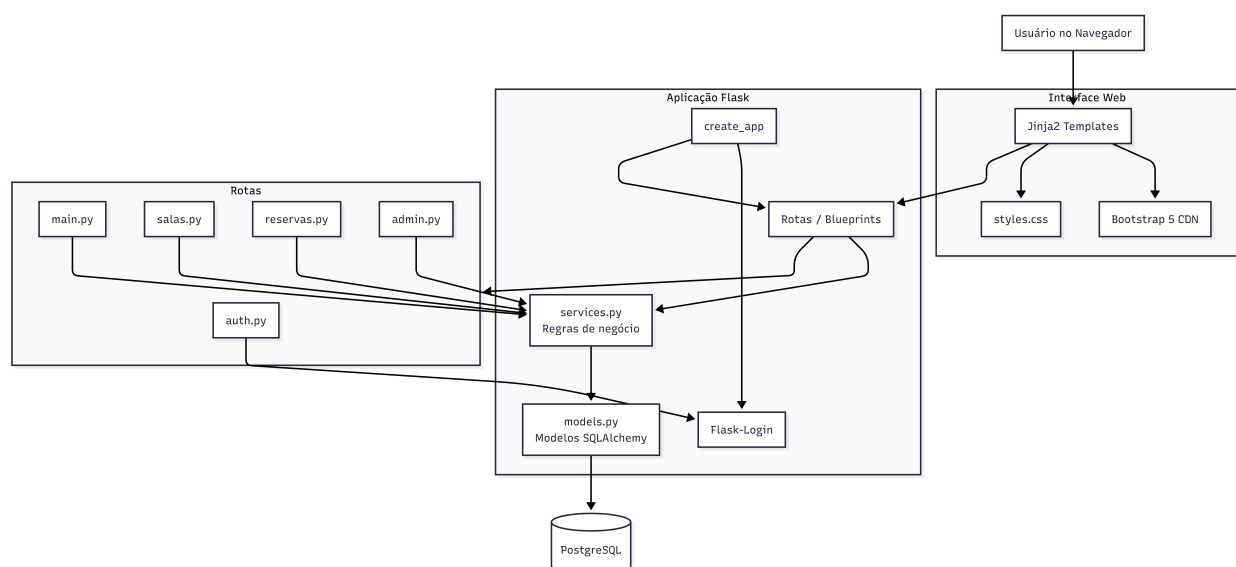


Figure 2: Arquitetura do sistema

3.2 Stack

- Python 3 e Flask.
- Flask-SQLAlchemy e PostgreSQL com psycopg.
- Flask-Login para autenticação.
- Jinja2 para templates.
- Bootstrap 5 por CDN.
- pytest para testes automatizados.

3.3 Justificativa da Stack

- Flask foi escolhido por permitir desenvolvimento rápido de aplicações web server-side.
- Flask-SQLAlchemy facilita o mapeamento entre classes Python e tabelas do PostgreSQL.
- Flask-Login simplifica autenticação, sessão e proteção de rotas.
- PostgreSQL foi adotado como banco relacional principal, mais adequado que SQLite para uma proposta próxima de implantação real.
- Jinja2 e Bootstrap 5 permitem criar telas responsivas sem necessidade de React, Node.js ou APIs externas.
- pytest permite validar regras de negócio de forma automatizada e repetível.

3.4 Organização

```
app/  
  routes/  
  templates/  
  static/  
  models.py  
  services.py  
  seed.py  
  auth_utils.py  
  extensions.py  
tests/  
docs/  
init_db.py  
run.py  
requirements.txt
```

3.5 Camadas

- Aplicação: cria a instância Flask, registra blueprints e configura extensões.
- Modelos: define entidades persistidas no banco.
- Rotas: organiza os fluxos de autenticação, dashboard, salas, reservas e administração.
- Serviços: centraliza regras de negócio de reservas, bloqueios, cancelamentos e configurações.
- Templates: renderiza as telas HTML com Jinja2 e Bootstrap.
- Testes: valida regras de negócio, rotas e renderização das principais telas.

3.6 Blueprints e Responsabilidades

Arquivo	Responsabilidade
<code>auth.py</code>	Controla login e logout dos usuários.
<code>main.py</code>	Renderiza o dashboard conforme o perfil autenticado.
<code>salas.py</code>	Lista salas, aplica filtros, considera bloqueios do dia atual e exibe horários disponíveis para reserva.
<code>reservas.py</code>	Lista reservas do aluno e processa cancelamentos.
<code>admin.py</code>	Concentra as funcionalidades administrativas: usuários, salas, exclusão controlada de salas, reservas, bloqueios e configurações.

3.7 Fluxo Geral de Requisição

O fluxo de uma requisição segue a estrutura tradicional de uma aplicação Flask:

1. O usuário acessa uma rota pelo navegador.
2. O Flask direciona a requisição ao blueprint responsável.
3. A rota valida autenticação e permissões quando necessário.
4. As regras de negócio são executadas nos serviços.
5. Os modelos SQLAlchemy consultam ou persistem dados no PostgreSQL.
6. A resposta é renderizada em template Jinja2 ou redirecionada para outra rota.

4 Modelo de Dados

4.1 Entidades

Entidade	Descrição
Usuario	Representa alunos e administradores. Armazena nome, e-mail, matrícula, senha com hash, perfil e status ativo.

Sala	Representa uma sala de estudo, com nome, localização, capacidade, descrição, status ativo e indicação de bloqueio.
Reserva	Representa uma reserva feita por um usuário para uma sala em uma data e horário. Possui finalidade, status, motivo de cancelamento e data de criação.
BloqueioSala	Representa bloqueios administrativos de sala por período, com motivo e status ativo.
ConfiguracaoSistema	Armazena configurações globais, como o limite máximo de reservas ativas por aluno.

4.2 Principais Campos

Tabela	Campo	Finalidade
usuarios	perfil	Define se o usuário atua como aluno ou administrador.
usuarios	senha_hash	Armazena a senha com hash seguro, evitando persistência de senha em texto puro.
salas	ativa	Indica se a sala está disponível para uso no sistema.
salas	bloqueada	Indica bloqueio geral da sala para manutenção.
reservas	status	Controla se a reserva está ativa ou cancelada.
reservas	finalidade	Registra o motivo acadêmico da reserva e aparece na consulta de horários ocupados.
reservas	motivo_cancelamento	Registra a justificativa do cancelamento.
bloqueios_sala	data e data_fim	Definem o período em que a sala ficará bloqueada.
configuracoes_sistema	chave e valor	Permite guardar configurações globais sem alterar código.

4.3 Relacionamentos

- Um usuário pode possuir várias reservas.
- Uma sala pode possuir várias reservas.
- Uma sala pode possuir vários bloqueios.
- Uma reserva pertence a um usuário e a uma sala.
- Um bloqueio pertence a uma sala.

4.4 Modelo Físico

O modelo físico representa as tabelas efetivamente persistidas pelo SQLAlchemy no PostgreSQL, incluindo chaves primárias, chaves estrangeiras e relacionamentos.

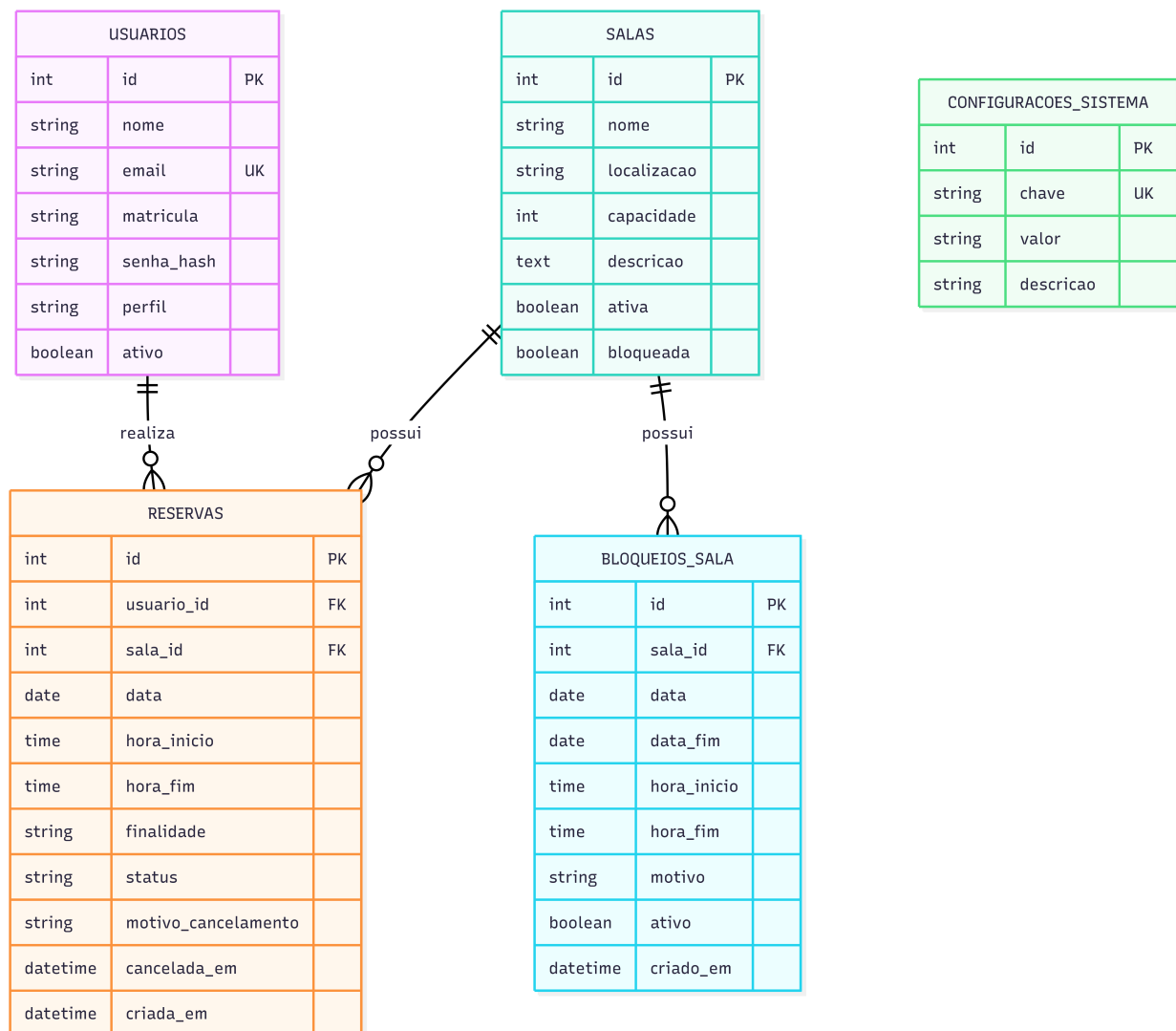


Figure 3: Modelo de dados do sistema Reserva de Salas de Estudo

4.5 Diagrama de Classes

O diagrama de classes representa as principais classes Python utilizadas pelo SQLAlchemy, incluindo atributos, propriedades e associações entre entidades. Ele é semelhante ao modelo de dados porque as classes do SQLAlchemy são utilizadas como base para criação das tabelas.

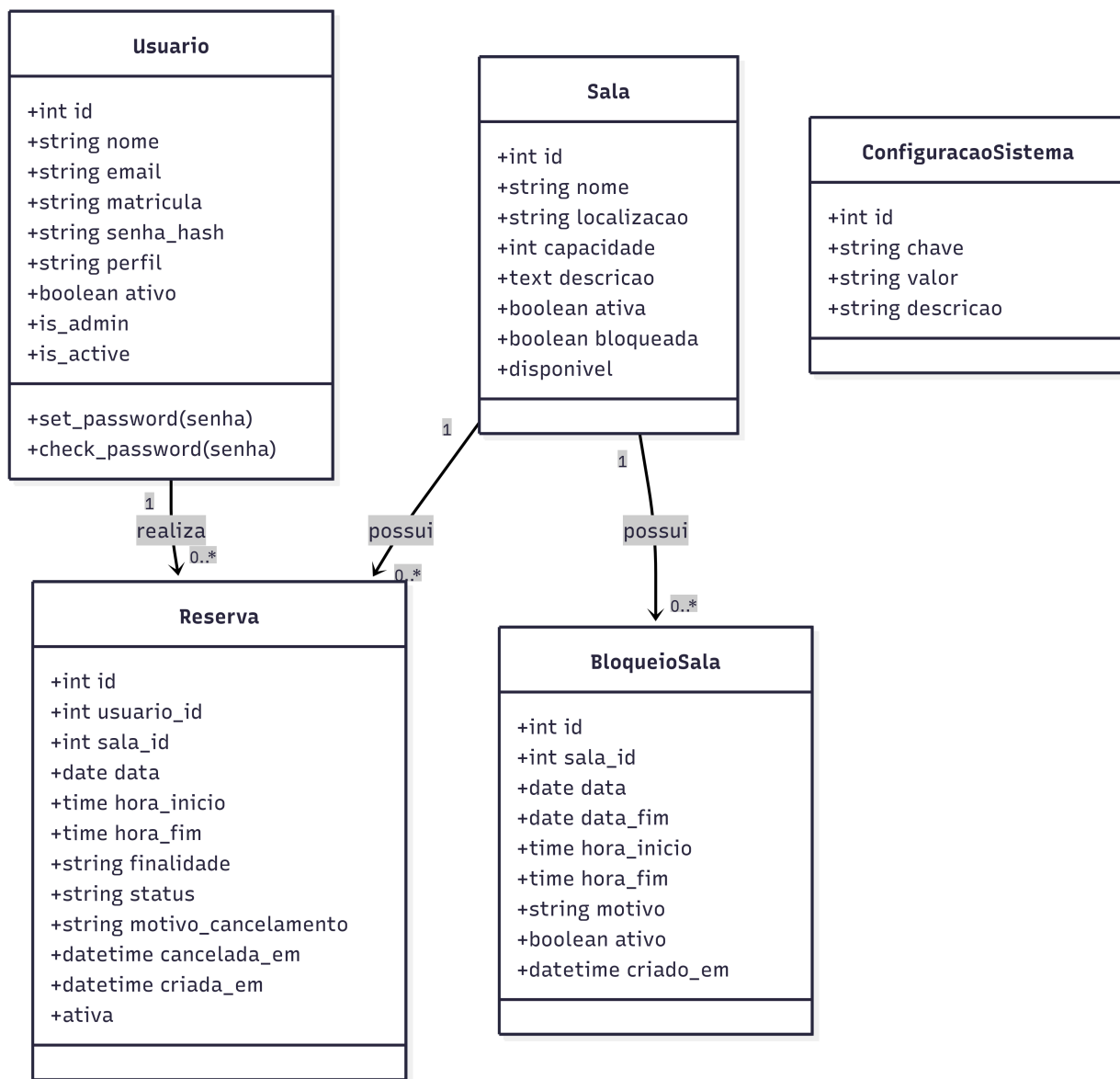


Figure 4: Diagrama de classes

4.6 Observações sobre Persistência

O sistema utiliza `db.create_all()` no script `init_db.py` para criar as tabelas quando ainda não existem. O mesmo script também executa ajustes simples de colunas e insere os dados iniciais por meio do `seed.py`. Para uma implantação de produção, seria recomendável utilizar uma ferramenta formal de migração, como Flask-Migrate/Alembic, mas isso ficou fora do escopo acadêmico definido.

5 Requisitos

5.1 Requisitos Funcionais

ID	Requisito
----	-----------

RF-01	Permitir login de usuários cadastrados.
RF-02	Permitir logout.
RF-03	Exibir dashboard conforme o perfil.
RF-04	Permitir consulta de salas.
RF-05	Permitir consulta de horários por sala e data, com blocos de uma hora.
RF-06	Permitir criação de reservas com finalidade.
RF-07	Permitir visualização das próprias reservas.
RF-08	Permitir cancelamento de reserva com motivo.
RF-09	Permitir cadastro de usuários pelo administrador.
RF-10	Permitir edição de usuários pelo administrador.
RF-11	Permitir ativar e desativar usuários.
RF-12	Permitir cadastro de salas.
RF-13	Permitir edição de salas.
RF-14	Permitir ativar, desativar e bloquear salas.
RF-15	Permitir visualização administrativa de reservas.
RF-16	Permitir filtro administrativo de reservas.
RF-17	Permitir cancelamento administrativo de reservas.
RF-18	Permitir bloqueio de salas por período.
RF-19	Permitir cancelamento de bloqueios.
RF-20	Permitir configuração do limite de reservas ativas por aluno.
RF-21	Exibir páginas 403 e 404.
RF-22	Permitir exclusão de salas sem reservas ou bloqueios registrados.
RF-23	Exibir bloqueios do dia atual na lista de salas e nos filtros de disponibilidade.

5.2 Priorização

Os requisitos essenciais foram aqueles necessários para demonstrar o fluxo completo de reserva de sala com controle de conflito e separação de perfis. Funcionalidades administrativas como gestão de usuários, salas, bloqueios, exclusão controlada de salas e limite de reservas foram incluídas para tornar o sistema demonstrável de ponta a ponta.

5.3 Requisitos Não Funcionais

ID	Requisito
RNF-01	Utilizar Python 3 e Flask.
RNF-02	Utilizar PostgreSQL como banco da aplicação.
RNF-03	Utilizar Flask-SQLAlchemy para persistência.
RNF-04	Utilizar Flask-Login para autenticação.
RNF-05	Utilizar Jinja2 para templates.
RNF-06	Utilizar Bootstrap 5 por CDN.
RNF-07	Ter layout responsivo.
RNF-08	Possuir testes automatizados com pytest.
RNF-09	Armazenar senhas com hash seguro.

RNF-10	Executar localmente sem Docker, React, Node.js, APIs externas ou nuvem.
--------	---

5.4 Regras de Negócio

ID	Regra
RN-01	A hora final deve ser posterior à hora inicial.
RN-02	Não aceitar reservas em datas passadas.
RN-03	A reserva deve ter duração fixa de uma hora.
RN-04	Uma sala não pode possuir reservas ativas sobrepostas na mesma data.
RN-05	Somente proprietário ou administrador pode cancelar reserva.
RN-06	Salas inativas ou bloqueadas não podem ser reservadas.
RN-07	Um aluno não pode criar reservas sobrepostas.
RN-08	Somente administradores gerenciam usuários, salas, bloqueios e configurações.
RN-09	Páginas protegidas exigem autenticação.
RN-10	Senhas devem ser armazenadas com hash seguro.
RN-11	Cancelamento de reserva deve registrar motivo.
RN-12	O administrador pode configurar o limite de reservas ativas.
RN-13	Bloqueios por período impedem reservas no intervalo.
RN-14	E-mails de usuários devem ser únicos.
RN-15	O administrador não pode desativar seu próprio usuário.
RN-16	Salas com reservas ou bloqueios registrados não podem ser excluídas, preservando o histórico do sistema.
RN-17	Bloqueios ativos para o dia atual devem ser refletidos na lista de salas e no contador de salas disponíveis.

5.5 Regras Implementadas no Backend

As validações críticas foram implementadas no backend, principalmente no arquivo `services.py`. Essa decisão evita que regras importantes dependam apenas da interface HTML. Entre as principais validações estão:

- impedimento de reservas em datas passadas;
- validação de duração fixa de uma hora;
- verificação de sobreposição de horários;
- bloqueio de reservas em salas inativas ou bloqueadas;
- limite de reservas ativas por aluno;
- obrigatoriedade de motivo no cancelamento.
- impedimento de exclusão de salas com histórico de reservas ou bloqueios.
- exibição de bloqueios por período na lista de salas quando atingem a data atual.

6 Matriz de Rastreabilidade

Req.	Regra	Rota	Teste
RF-01	RN-09, RN-10	/login	test_login_valido; test_login_invalido
RF-03	RN-09	/dashboard	test_templates_principais_do_aluno_renderizam
RF-04	RN-09	/salas/	test_filtro_de_salas_por_capacidade
RF-05	RN-06	/salas/horarios	test_horarios_disponiveis_renderiza
RF-06	RN-01 a RN-04, RN-06, RN-07, RN-12, RN-13	/salas/horarios	testes de criação e rejeição de reserva
RF-08	RN-05, RN-11	/reservas/{id}/cancelar	test_cancelamento_de_reserva
RF-09	RN-08, RN-10, RN-14	/admin/usuarios/novo	test_admin_cadastra_usuario_aluno
RF-10	RN-08, RN-10, RN-14, RN-15	/admin/usuarios/{id}/editar	test_admin_edita_usuario_sem_alterar_senha
RF-12	RN-08	/admin/salas/nova	test_admin_cadastra_sala_valida
RF-22	RN-08, RN-16	/admin/salas/{id}/excluir	test_admin_exclui_sala_sem_historico; test_admin_nao_exclui_sala_com_reserva
RF-18	RN-08, RN-13	/admin/bloqueios	test_admin_cria_bloqueio_por_periodo
RF-20	RN-08, RN-12	/admin/configuracoes	test_admin_atualiza_limite_de_reservas
RF-21	RN-08, RN-09	/admin/	test_bloqueio_acesso_administrativo_para_aluno
RF-23	RN-13, RN-17	/salas/	test_lista_salas_mostra_bloqueio_do_dia_atual

6.1 Análise da Cobertura

A matriz mostra que os requisitos centrais possuem ligação com regras de negócio, rotas e testes automatizados. Os testes cobrem os fluxos críticos de autenticação, autorização, criação de reserva, rejeições de regras inválidas, cancelamento, gestão administrativa, bloqueios, exclusão controlada de salas e configuração de limite.

Aspectos visuais, como responsividade, disposição dos elementos e experiência de navegação, foram tratados por inspeção manual durante a demonstração, pois são menos adequados para a suíte automatizada atual.

7 Gerenciamento de Riscos

ID	Risco	Prob.	Impacto	Mitigação e resposta
R-01	PostgreSQL não instalado ou iniciado	Média	Alto	Documentar instalação, iniciar serviço e validar DATABASE_URL.
R-02	Erro de conexão com banco	Média	Alto	Conferir usuário, senha, host, porta e banco.
R-03	Conflito de reservas não detectado	Baixa	Alto	Centralizar regras em serviços e manter testes regressivos.
R-04	Aluno acessar área administrativa	Baixa	Alto	Aplicar login_required e admin_required.

R-05	Senha armazenada de forma insegura	Baixa	Alto	Usar hash seguro via Werkzeug.
R-06	Escopo crescer além do planejado	Média	Médio	Registrar exclusões e priorizar funcionalidades essenciais.
R-07	Falha na demonstração	Média	Alto	Manter seed, README, testes e banco inicializados.
R-08	Templates quebrados após alteração de rota	Média	Médio	Testar renderização das principais telas.
R-09	E-mail duplicado no cadastro de usuário	Média	Médio	Validar unicidade no back-end.
R-10	Documentação desatualizada	Média	Médio	Revisar artefatos após mudanças de escopo.

7.1 Estratégia de Resposta

Os riscos de maior impacto estão relacionados ao banco de dados, à integridade das reservas e ao controle de acesso. Para esses pontos, a estratégia adotada foi preventiva: documentação de comandos, validações centralizadas em serviços, testes automatizados e uso de decorators de autenticação e autorização.

Riscos ligados à apresentação e documentação foram tratados com geração de artefatos finais em PDF, arquivo `comandosFINAL` e arquivo `validacaoFINAL`.

8 Segurança e Controle de Acesso

8.1 Autenticação

A autenticação é feita com Flask-Login. Usuários precisam informar e-mail e senha. A senha armazenada no banco não fica em texto puro; ela é transformada em hash por funções do Werkzeug.

8.2 Autorização

As rotas administrativas usam o decorator `admin_required`. Esse controle impede que alunos acessem funcionalidades de gestão, como cadastro de usuários, cadastro de salas, bloqueios, reservas administrativas e configurações.

8.3 Proteção de Dados

O sistema não implementa criptografia avançada, auditoria completa ou LGPD em nível de produção, pois esses pontos estão fora do escopo acadêmico definido. Ainda assim, evita o principal risco imediato ao não armazenar senhas em texto puro.

9 Testes e Qualidade

9.1 Estratégia de Testes

Os testes automatizados utilizam pytest e o cliente de teste do Flask. A suíte usa banco isolado em memória para executar rapidamente e não depender do PostgreSQL local durante a validação das regras.

9.2 Cobertura Funcional

Os testes validam login, erro de login, proteção de rotas, bloqueio de acesso administrativo para aluno, cadastro e edição de usuários, criação de reservas, rejeição de reservas inválidas, cancelamentos, bloqueios, configuração de limite, exclusão controlada de salas, listagem de salas bloqueadas no dia atual e renderização das principais telas.

9.3 Resultado

A última execução registrada apresentou:

36 passed

10 Implantação Local

10.1 Banco de Dados

O banco principal da aplicação é PostgreSQL. A URL padrão configurada no projeto é:

```
postgresql+psycopg://reserva_salas:reserva_salas@localhost:5432/reserva_salas
```

Caso o ambiente local utilize outro usuário, senha ou banco, a variável `DATABASE_URL` deve ser definida antes da execução.

10.2 Comandos Principais

```
sudo systemctl start postgresql
source .venv/bin/activate
python init_db.py
pytest
python run.py
```

10.3 Dados Iniciais

O script de inicialização cria o administrador, alunos de demonstração, salas iniciais e configuração padrão de limite de reservas ativas por aluno.

11 Limitações e Evolução

11.1 Limitações do Sistema

- Não há envio de notificações por e-mail.

- Não há integração com sistema acadêmico real.
- Não há cadastro público de usuários.
- Não há painel de relatórios estatísticos avançados.
- Não há migrações formais com Alembic.

11.2 Possíveis Evoluções

- Adicionar Flask-Migrate para controle formal de mudanças no banco.
- Criar relatórios de ocupação por sala e período.
- Adicionar paginação nas listas administrativas.
- Criar filtros avançados para usuários.
- Adicionar notificações por e-mail para reserva e cancelamento.
- Criar auditoria administrativa para alterações sensíveis.

12 Conclusão

A documentação técnica consolida a estrutura do sistema, seus requisitos, regras de negócio, rastreabilidade, riscos, estratégia de testes e orientações de implantação local. O sistema possui escopo coerente com a proposta acadêmica, separação clara de responsabilidades, banco relacional, autenticação por perfil, validações no backend e testes automatizados suficientes para apoiar a demonstração e homologação.